

AI Brand Brain

Complete Product Implementation Guide

This document explains:

- what this product is
- what problem it solves
- which technologies are used
- which file does what
- how the full flow works
- how to run it
- how to extend it
- what is still mock/MVP vs what is production-ready

1. Product Vision

AI Brand Brain is an AI-driven marketing automation platform for brands, founders, and businesses.

The intended user flow is:

- 1 create a brand
- 2 add products
- 3 upload product images or videos
- 4 generate captions, scripts, and video campaigns using AI
- 5 render marketing video assets
- 6 connect social channels
- 7 auto-schedule and publish content
- 8 track performance
- 9 compare which creative variant performs best

The long-term goal is:

- one brand setup
- many automatically generated creatives
- automated posting
- analytics-based optimization

2. Core Problem This Product Solves

Small and mid-sized brands usually face these issues:

- content creation is repetitive
- video creation takes too much time
- every platform needs different formatting
- posting manually is slow
- analytics are scattered
- teams do not know which creative angle is actually working

AI Brand Brain solves this by turning product inputs into an automated campaign system.

3. Tech Stack

Frontend

- React 18
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui style components
- Axios

Backend

- FastAPI
- SQLAlchemy
- Pydantic
- APScheduler

Database

- PostgreSQL intended for production
- SQLite also works for simple local dev/testing

AI / Media

- OpenAI for copy/video campaign generation

- OpenAI TTS fallback for narration when available
- Pillow for frame generation
- imageio + imageio-ffmpeg for MP4 rendering
- Cloudinary optional for media storage

4. Current Product Capabilities

Implemented

- authentication
- brand management
- product management
- product media upload
- AI content generation
- AI video campaign generation
- batch creative variant generation
- MP4 render pipeline from uploaded media + generated overlays
- scheduling posts
- campaign auto-scheduler for multiple variants
- mock-safe social posting in development/demo mode
- performance analytics generation
- top creative comparison

Partially Implemented / MVP

- real social posting requires valid platform tokens
- narration works best through OpenAI TTS when configured
- analytics are generated heuristically for MVP feedback unless real platform sync is added

Not Fully Implemented Yet

- full OAuth login for Instagram / YouTube / LinkedIn
- real platform analytics ingestion
- advanced AI cinematic generation like Runway/Pika
- real long-term machine-learning optimizer
- billing / subscriptions / multi-team workflows

5. Project Structure

Root

- ``README.md``
- ``MIGRATION.md``
- ``env.example``
- ``docker-compose.yml``
- ``PRODUCT_IMPLEMENTATION_GUIDE.md``

Backend

- ``backend/app/main.py``
- FastAPI app entry
- router registration
- scheduler startup
- uploads mount
- ``backend/app/database.py``
- SQLAlchemy engine/session setup
- PostgreSQL + psycopg normalization
- SQLite-safe local configuration
- ``backend/app/core/config.py``
- environment variables and settings
- ``backend/app/core/security.py``
- auth helpers, JWT, current user

Backend Models

- ``backend/app/models/user.py``
- user accounts
- ``backend/app/models/brand.py``
- brand details
- ``backend/app/models/product.py``
- product details, benefits, features

- ``backend/app/models/media.py``
- uploaded product media
- ``backend/app/models/content.py``
- generated text content
- ``backend/app/models/video.py``
- generated video campaign projects
- ``backend/app/models/social.py``
- connected accounts
- scheduled posts
- ``backend/app/models/analytics.py``
- post analytics
- summary models

Backend Routers

- ``backend/app/routers/auth.py``
- register/login/me
- ``backend/app/routers/brands.py``
- brand CRUD
- logo upload
- ``backend/app/routers/products.py``
- product CRUD
- media upload/list/delete
- ``backend/app/routers/content.py``
- text content generation
- ``backend/app/routers/videos.py``
- single video generation
- batch variant generation
- rendering
- auto-scheduling campaign variants

- ``backend/app/routers/social.py``
- connect accounts
- schedule posts
- publish now

- ``backend/app/routers/analytics.py``
- summary analytics
- timeline analytics
- top creatives

Backend Services

- ``backend/app/services/ai_service.py``
- text content generation

- ``backend/app/services/video_service.py``
- creative angles
- campaign generation
- image-to-video rendering
- MP4 generation
- voiceover synthesis fallback logic

- ``backend/app/services/media_service.py``
- Cloudinary/local media upload

- ``backend/app/services/social_service.py``
- real social publish integrations
- mock-safe posting mode

- ``backend/app/services/analytics_service.py``
- generated analytics after publishing

- ``backend/app/services/scheduler.py``
- automatic scheduled publishing loop

Frontend

- ``frontend/src/App.tsx``
- routes and auth guard

- ``frontend/src/contexts/AuthContext.tsx``
- global auth handling
- ``frontend/src/lib/api.ts``
- all API client methods

Frontend Pages

- ``frontend/src/pages/Auth.tsx``
- login/signup UI
- ``frontend/src/pages/Dashboard.tsx``
- high-level business overview
- ``frontend/src/pages/Generate.tsx``
- text/caption/script generation
- ``frontend/src/pages/Videos.tsx``
- video campaign generation
- batch variant generation
- render controls
- auto-scheduling campaign variants
- ``frontend/src/pages/Products.tsx``
- product CRUD and media uploads
- ``frontend/src/pages/Schedule.tsx``
- queue individual posts
- ``frontend/src/pages/Analytics.tsx``
- performance insights
- top creatives
- ``frontend/src/pages/Settings.tsx``
- brand settings
- social account connection

6. End-to-End Product Flow

6.1 Authentication

User signs up or logs in through:

- ``frontend/src/pages/Auth.tsx``
- ``backend/app/routers/auth.py``

JWT token is stored in local storage and attached to API requests through:

- ``frontend/src/lib/api.ts``

6.2 Brand Setup

User creates a brand from Settings.

Stored data includes:

- brand name
- description
- industry
- tone
- target audience
- values
- website
- logo

Files involved:

- ``frontend/src/pages/Settings.tsx``
- ``backend/app/routers/brands.py``
- ``backend/app/models/brand.py``

6.3 Product Setup

User creates products and uploads images/videos.

Files involved:

- ``frontend/src/pages/Products.tsx``

- ``backend/app/routers/products.py``
- ``backend/app/services/media_service.py``
- ``backend/app/models/product.py``
- ``backend/app/models/media.py``

Media can be:

- stored locally in ``uploads/``
- uploaded to Cloudinary if configured

6.4 Text Content Generation

User can generate:

- captions
- scripts
- hashtags
- video scripts

Files involved:

- ``frontend/src/pages/Generate.tsx``
- ``backend/app/routers/content.py``
- ``backend/app/services/ai_service.py``

If no OpenAI key is configured:

- mock content is returned

6.5 Video Campaign Generation

Video generation is not just “make video”.

The app first creates a campaign package:

- hook
- storyboard
- scenes
- voiceover script

- caption
- hashtags
- shot plan

Files involved:

- ``frontend/src/pages/Videos.tsx``
- ``backend/app/routers/videos.py``
- ``backend/app/services/video_service.py``
- ``backend/app/models/video.py``

Single Generation

Endpoint:

- ``POST /api/videos/generate``

Batch Variant Generation

Endpoint:

- ``POST /api/videos/generate-batch``

Creative angles currently include things like:

- pain-point solution
- before/after
- premium lifestyle
- founder credibility
- social proof
- urgency

This is how the system creates multiple “random but structured” ad variants.

6.6 Video Rendering

After campaign generation, the app can render a real video asset.

Current Renderer

Renderer uses:

- uploaded images
- generated scenes
- overlay text
- generated voiceover script

It outputs:

- MP4 render
- preview asset

Files involved:

- ``backend/app/services/video_service.py``

Voiceover

The renderer tries:

- 1 OpenAI TTS if configured
- 2 Windows speech fallback where available

Current Limitations

This is a template-style renderer, not cinematic generative video.

It is still very useful for:

- MVP product videos
- ad concept validation
- quick campaign exports

6.7 Auto-Scheduling Campaign Variants

This is one of the strongest features currently built.

User can:

- 1 generate multiple variants
- 2 select variants
- 3 choose a connected account

- 4 choose start date/time
- 5 choose spacing interval
- 6 auto-schedule the full test campaign

Files involved:

- ``frontend/src/pages/Videos.tsx``
- ``backend/app/routers/videos.py``

Endpoint:

- ``POST /api/videos/auto-schedule``

6.8 Social Posting

The scheduler and social services handle posting.

Files involved:

- ``backend/app/services/scheduler.py``
- ``backend/app/services/social_service.py``
- ``backend/app/routers/social.py``

Supported Platforms

- Instagram
- YouTube
- LinkedIn

Posting Modes

Real Mode

If valid tokens are configured and production setup exists:

- real APIs are called

Mock / Demo Mode

If app is in development/demo or token looks like mock/demo:

- system generates a mock post URL
- post is marked posted
- analytics can still be generated

This is useful for demos, testing, and sales use.

6.9 Analytics and “Is this ad working?”

After posting, analytics are generated/stored.

Files involved:

- ``backend/app/services/analytics_service.py``
- ``backend/app/routers/analytics.py``
- ``frontend/src/pages/Analytics.tsx``

What the UI Shows

- views
- likes
- comments
- shares
- engagement rate
- top platform
- recommendations
- top creative variants

Current Analytics Mode

Today the app uses generated heuristics for MVP evaluation unless real platform sync is implemented.

That means:

- good for testing and demos
- good for internal decision support
- not yet true ad-manager-grade analytics

7. Automatic Optimization Logic

Right now optimization is implemented at MVP level through:

- multiple creative-angle generation
- performance attribution
- top creative leaderboard
- recommendations in analytics summary

This means the system already helps answer:

- which ad concept is better?
- which platform is best?
- should we double down on a certain creative angle?

What is not yet built:

- retraining or model fine-tuning
- automatic creative regeneration based on historical winners

8. Environment Variables

Root ``.env.example`` and backend ``.env.example`` are starting points.

Important variables:

- ``DATABASE_URL``
- ``SECRET_KEY``
- ``OPENAI_API_KEY``
- ``CLOUDINARY_CLOUD_NAME``
- ``CLOUDINARY_API_KEY``
- ``CLOUDINARY_API_SECRET``
- ``INSTAGRAM_APP_ID``
- ``INSTAGRAM_APP_SECRET``
- ``YOUTUBE_CLIENT_ID``
- ``YOUTUBE_CLIENT_SECRET``
- ``LINKEDIN_CLIENT_ID``
- ``LINKEDIN_CLIENT_SECRET``
- ``FRONTEND_URL``
- ``ENVIRONMENT``

Recommended Local Dev Setup

For easiest local running:

- use SQLite
- use `ENVIRONMENT=development`
- set `OPENAI\_API\_KEY`

Example:

```
DATABASE_URL=sqlite:///./app.db
SECRET_KEY=change-this-to-a-long-random-secret
OPENAI_API_KEY=your_openai_api_key
FRONTEND_URL=http://localhost:5173
ENVIRONMENT=development
```

9. How to Run the Project

Option A: Docker

From root:

```
docker compose up --build
```

Open:

- frontend: `http://localhost`
- docs: `http://localhost:8000/docs`

Option B: Local

Backend

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
```

Frontend URL:

- ``http://localhost:5173``

Backend docs:

- ``http://localhost:8000/docs``

10. How to Implement This in Real Production

If you want to implement this as a production business product, do it in phases:

Phase 1: Core SaaS Stability

- clean deployments
- separate dev/staging/prod envs
- proper migrations
- remove local mock assumptions where needed
- set up PostgreSQL in production

Phase 2: Real Media and Video Quality

- stronger render engine
- subtitles
- background music
- better transitions
- richer templates

Phase 3: Real Social Integrations

- full OAuth flow
- secure token refresh
- webhook support
- actual analytics ingestion

Phase 4: Real Optimizer

- campaign performance history
- creative scoring engine
- best-time prediction
- angle recommendation engine
- automated next-batch generation from winners

11. Important Implementation Notes

Video Rendering

Current render pipeline is best for:

- ad prototypes
- simple social video creation
- testing multiple creative concepts quickly

It is not yet equivalent to:

- Runway
- Pika
- full cinematic AI video engines

Posting

Current posting logic supports:

- real API integration path
- mock/demo-safe publishing path

That means product demos remain usable even without full platform onboarding.

Analytics

Current analytics are MVP-grade:

- good enough to compare creatives
- not yet ad-platform source-of-truth analytics

12. Recommended Next Improvements

If continuing development, highest-value next steps are:

- 1 real OAuth social login flows
- 2 real platform analytics sync
- 3 subtitle and music support in renderer

- 4 auto-pick best posting time from historical performance
- 5 regenerate next campaign batch from winning angle
- 6 content calendar view
- 7 billing and subscription plans

13. File-to-Feature Mapping Summary

Auth

- ``frontend/src/pages/Auth.tsx``
- ``backend/app/routers/auth.py``

Brand Setup

- ``frontend/src/pages/Settings.tsx``
- ``backend/app/routers/brands.py``

Product & Media

- ``frontend/src/pages/Products.tsx``
- ``backend/app/routers/products.py``
- ``backend/app/services/media_service.py``

Text Generation

- ``frontend/src/pages/Generate.tsx``
- ``backend/app/services/ai_service.py``

Video Campaign Generation

- ``frontend/src/pages/Videos.tsx``
- ``backend/app/routers/videos.py``
- ``backend/app/services/video_service.py``

Batch Variant Testing

- ``frontend/src/pages/Videos.tsx``
- ``backend/app/routers/videos.py``

Campaign Auto Scheduler

- ``frontend/src/pages/Videos.tsx``
- ``backend/app/routers/videos.py``

Social Posting

- ``frontend/src/pages/Schedule.tsx``
- ``backend/app/routers/social.py``
- ``backend/app/services/social_service.py``
- ``backend/app/services/scheduler.py``

Analytics

- ``frontend/src/pages/Analytics.tsx``
- ``backend/app/routers/analytics.py``
- ``backend/app/services/analytics_service.py``

14. Final Product Status

This product is now a strong launchable MVP with these real capabilities:

- multi-step marketing workflow
- media-aware AI generation
- video campaign generation
- variant testing
- scheduling
- posting
- analytics
- campaign comparison

It is not yet a fully enterprise-hardened autonomous marketing OS, but the core product direction is now real, visible, and usable.